

## DAY 7

### Task1

- Junit

```
package Junit;

public class DemoCalculator {

    public int add(int a, int b) {
        return a + b;
    }
}
```

```
package Junit;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class Day7_TestCases {

    @Test
    void testAddition() {
        Junit.DemoCalculator calc = new Junit.DemoCalculator();
        assertEquals(15, calc.add(10, 5));
    }

    @Test
    void testAdditionFail() {
        Junit.DemoCalculator calc = new Junit.DemoCalculator();

        assertEquals(20, calc.add(10, 5),
            "Expected 20 but actual is 15");
    }
}
```

### Task 2

#### Exception testing

```
package Junit;

public class Day7_BankCustomer {

    private String accountNumber;
    private double balance;

    public Day7_BankCustomer(String accountNumber, double balance) {
        this.accountNumber = accountNumber;
        this.balance = balance;
    }

    public void withdraw(double amount) {
        if (amount > balance) {
            throw new IllegalArgumentException("Insufficient balance");
        }
    }
}
```

```

    }
    balance -= amount;
}

public double getBalance() {
    return balance;
}
}

```

```

package Junit;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

public class Dat7_Exception_testing {

    @Test
    void testWithdrawInsufficientFunds() {

        Day7_BankCustomer account =
            new Day7_BankCustomer("123", 100.0);

        Exception ex = assertThrows(
            IllegalArgumentException.class,
            () -> account.withdraw(200.0)
        );

        assertEquals("Insufficient balance", ex.getMessage());
    }
}

```

### Task 3

For above code create fail test cases

```

package Junit;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

public class Dat7_Exception_testing {

    @Test
    void testWithdrawInsufficientFunds() {

        Day7_BankCustomer account =
            new Day7_BankCustomer("123", 100.0);

        Exception ex = assertThrows(
            IllegalArgumentException.class,
            () -> account.withdraw(200.0)
        );

        assertEquals("Insufficient balance", ex.getMessage());
    }
}

```

```

}

@Test
void testWithdrawInsufficientFundsFail() {

    Day7_BankCustomer account =
        new Day7_BankCustomer("123", 100.0);

    Exception ex = assertThrows(
        IllegalArgumentException.class,
        () -> account.withdraw(200.0)
    );
    assertEquals("Low balance", ex.getMessage());
}
}

```

#### Task 04:

Create a java code for the Parameterised Test case

```

package Junit;

public class Day7_Palindrome {

    public boolean isPalindrome(String str) {
        String reversed = new StringBuilder(str).reverse().toString();
        return str.equalsIgnoreCase(reversed);
    }
}

```

```

package Junit;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;

import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.junit.jupiter.api.Assertions.assertTrue;

class Day7_Palindrome_testing {

    Day7_Palindrome palindrome = new Day7_Palindrome();

    @ParameterizedTest
    @ValueSource(strings = {"amma", "mom", "nitin"})
    void testPalindromePass(String candidate) {
        assertTrue(palindrome.isPalindrome(candidate));
    }

    @Test
    void testPalindromeFail() {
        assertTrue(
            palindrome.isPalindrome("hello"),
            "Expected true but got false"
        );
    }
}

```

```

@ParameterizedTest
@ValueSource(strings = {"java", "spring", "bank"})
void testNotPalindrome(String candidate) {
    assertFalse(palindrome.isPalindrome(candidate));
}
}

```

#### Task 05:

What is Junit Life cycle hook (set up tear down) ?

- It is a special method that execute before and after test cases. They help prepare the test environment, initialize required objects, and clean up resources once the test execution is complete.
- In JUnit 5, @BeforeAll runs once before all test methods, @BeforeEach runs before every test case, @AfterEach runs after every test case, and @AfterAll runs once after all tests have finished. These hooks ensure that each test runs independently with a consistent setup and proper cleanup, making the testing process more reliable and maintainable.

#### Task 06

Create a java code for life cycle hook setup/ tear down test file

```

package Junit;

import java.util.HashMap;
import java.util.Map;

public class Day7_DatabaseService {

    private Map<String, String> database;

    public void connect() {
        database = new HashMap<>();
        System.out.println("Database Connected");
    }

    public void disconnect() {
        database.clear();
        System.out.println("Database Disconnected");
    }

    public void insert(String key, String value) {
        database.put(key, value);
    }

    public String fetch(String key) {
        return database.get(key);
    }
}

```

```

package Junit;

import org.junit.jupiter.api.*;

```

```

import static org.junit.jupiter.api.Assertions.*;

public class Day7_LifeCycleHooks_Setup_TearDown {

    private Junit.Day7_DatabaseService db;

    @BeforeEach
    void init() {
        db = new Junit.Day7_DatabaseService();
        db.connect();
    }

    @AfterEach
    void cleanup() {
        db.disconnect();
    }

    @Test
    void testInsertAndFetchPass() {
        db.insert("user1", "Prasunamba");
        assertEquals("Prasunamba", db.fetch("user1"));
    }

    @Test
    void testInsertAndFetchFailValue() {
        db.insert("user1", "Prasunamba");

        assertEquals(
            "WrongName",
            db.fetch("user1"),
            "Expected WrongName but got " + db.fetch("user1")
        );
    }

    @Test
    void testFetchNonExistentKeyFail() {
        assertEquals(
            "SomeValue",
            db.fetch("missingKey"),
            "Expected SomeValue but got null"
        );
    }
}

```

## Task 07

What are assertions in JUnit 5.. Explain

- Assertions in JUnit 5 are methods used to verify whether the actual output of a program matches the expected output during test execution. They play a vital role in unit testing by helping developers validate the correctness of their code.
- JUnit 5 provides various assertion methods such as
  - assertEquals() to compare expected and actual values,
  - assertTrue() and assertFalse() to check boolean conditions
  - assertNull() and assertNotNull() to verify object references
  - assertThrows() to ensure that a specific exception is thrown.

- By using assertions, testers can automatically check application behavior and quickly identify defects, making the testing process more reliable and efficient.

### Task 08

What is time test performance service means 👍 ?

- Time Test Performance Service in JUnit refers to testing whether a piece of code completes its execution within a specified time limit.
- JUnit 5 provides the `assertTimeout()` assertion for this purpose. If the code executes within the defined time, the test passes otherwise, it fails. Time-based testing is useful for validating the efficiency of methods, APIs, database operations, and other services where response time is important.

### Task 09

For the given timeout test Performance Service can you create a java code

```
package Junit;
public class Day7_PerformanceService {

    public void quickOperation() throws InterruptedException {
        Thread.sleep(100); // 100 ms
    }

    public void slowOperation() throws InterruptedException {
        Thread.sleep(1000); // 1000 ms (1 second)
    }
}
```

```
package Junit;

import org.junit.jupiter.api.Test;
import java.time.Duration;
import static org.junit.jupiter.api.Assertions.assertTimeout;

class Day7_TimeoutTest_PerformanceCheck {

    @Test
    void testQuickExecutionPass() {

        Junit.Day7_PerformanceService service =
            new Junit.Day7_PerformanceService();

        assertTimeout(Duration.ofMillis(500), () -> {
            service.quickOperation(); // completes in 100ms
        });
    }

    @Test
    void testSlowExecutionFail() {

        Junit.Day7_PerformanceService service =
            new Junit.Day7_PerformanceService();
    }
}
```

```

    assertTimeout(Duration.ofMillis(500), () -> {
        service.slowOperation(); // takes 1000ms
    });
}

@Test
void testUnrealisticTimeoutFail() {

    Junit.Day7_PerformanceService service =
        new Day7_PerformanceService();

    assertTimeout(Duration.ofMillis(50), () -> {
        service.quickOperation(); // takes 100ms
    });
}
}

```

#### Task 10:

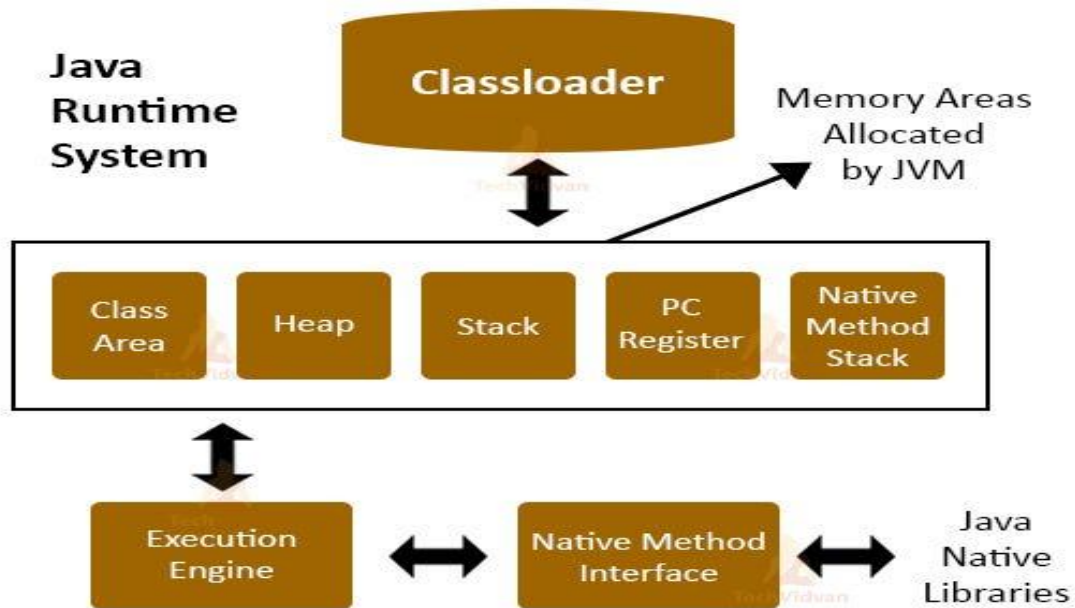
Now what do you think are the different types of JUnit 5 tests we have ... list them down

- JUnit 5 provides different types of tests to verify various aspects of an application. These include unit tests, parameterized tests, repeated tests, and dynamic tests for validating code under different conditions.
- It also supports nested, disabled, and conditional tests to improve test organization and flexibility. Exception and timeout tests help ensure correct error handling and performance. Additionally, lifecycle methods and integration tests make it easier to manage test execution and verify interactions between application components.

#### Task 11:

Write about JVM architecture using a diagram.

# JVM Architecture



The Java Virtual Machine (JVM) is a program that runs Java applications and allows them to work on different operating systems. Its Class Loader loads Java class files into memory, while the Memory stores objects, variables, and method information during execution. The Execution Engine reads and executes the Java bytecode. The Garbage Collector automatically removes unused objects to free up memory. The Java Native Interface (JNI) allows Java programs to communicate with code written in other languages.

## Task 12:

Why are enums useful?

- Makes code easier to understand and maintain.
- Allows only predefined values and prevents invalid assignments.
- Errors are caught during compilation rather than at runtime.
- Adding or modifying constants is easier and safer.
- Suitable for statuses, roles, days, months, payment types, etc.

## Task 13

Wap to show the importance of enums in real world banking application

```
enum AccountType {  
    SAVINGS,  
    CURRENT,  
    FIXED_DEPOSIT  
}  
  
public class BankingApplication {  
    public static void main(String[] args) {  
        AccountType account = AccountType.SAVINGS;  
        System.out.println("Banking Application");  
    }  
}
```



```

switch (account) {
    case SAVINGS:
        System.out.println("Account Type: Savings Account");
        System.out.println("Interest Rate: 4% per annum");
        break;

    case CURRENT:
        System.out.println("Account Type: Current Account");
        System.out.println("No interest is provided.");
        break;

    case FIXED_DEPOSIT:
        System.out.println("Account Type: Fixed Deposit Account");
        System.out.println("Interest Rate: 7% per annum");
        break;
}
}
}

```

#### Task 14

List down all the string handling functions with example in a single java code.. With respect to banking application

### String Functions Used

- length() – Returns the length of the string.
- trim() – Removes leading and trailing spaces.
- toUpperCase() – Converts the string to uppercase.
- substring() – Extracts part of the string.
- startsWith() – Checks whether the string starts with a specified prefix.
- contains() – Checks whether the string contains a specified text.
- replace() – Replaces one substring with another.

```

public class BankingStringDemo {
    public static void main(String[] args) {

        String bankName = "State Bank of India";
        String customerName = " Monika M ";
        String accountNo = "SBI9845678";

        System.out.println("Bank Name Length: " + bankName.length());
        System.out.println("Customer Name: " + customerName.trim());
        System.out.println("Bank Name in Upper Case: " + bankName.toUpperCase());
        System.out.println("Account Number: " + accountNo.substring(3));
        System.out.println("Account starts with SBI: " + accountNo.startsWith("SBI"));
        System.out.println("Bank name contains 'India': " + bankName.contains("India"));
        System.out.println("Updated Bank Name: " + bankName.replace("India", "Limited"));
    }
}

```

}  
}